National Economics University
College of Technology — DATCOM Lab

Project 22: Panic Selling Detection for Individual Investors

Student Name(s)

Student ID(s)

Supervisor: Hung Tran

Course: Introduction to Database

Class: DS66B

Spring 2026

Abstract

This report presents the design and implementation of a data-driven system for detecting and quantifying panic selling behavior among individual investors in the Vietnamese stock market. The system integrates a relational database built on MySQL with a Python-based behavioral analytics pipeline to identify abnormal selling patterns under market stress conditions.

The project generates a synthetic dataset simulating three investor archetypes — FOMO, Rational, and Noise investors — trading across nine Vietnamese stock tickers over the year 2025. Behavioral features including drawdown level, sell spike intensity, and loss sensitivity are extracted from transaction histories and aggregated into a Panic Score for each investor. Advanced database objects including indexes, views, stored procedures, user-defined functions, and triggers are implemented to support efficient querying and automated warning generation.

A lightweight web application built with Flask provides an interactive interface for querying investor profiles, viewing panic scores, and monitoring behavioral warnings in real time.

Keywords: panic selling, behavioral finance, synthetic data generation, MySQL, Python, investor profiling, market stress detection

Contents

Abstract i

List of Figures iv

List of Tables v

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Project Objectives . . . 1
- 1.3 Scope . . . 1
- 1.4 Report Structure . . . 2

2 System Analysis and Requirements 3
- 2.1 Problem Statement . . . 3
- 2.2 Functional Requirements . . . 3
- 2.3 Non-Functional Requirements . . . 3
- 2.4 System Architecture Overview . . . 4
- 2.5 Market Regime Classification . . . 4

3 Database Design and Implementation 5
- 3.1 Entity-Relationship Diagram . . . 5
- 3.2 Relational Schema . . . 5
  - 3.2.1 Investors . . . 5
  - 3.2.2 Market_Data . . . 5
  - 3.2.3 Transactions . . . 6
  - 3.2.4 Portfolio_History . . . 6
  - 3.2.5 BehaviorSignals . . . 6
  - 3.2.6 Warnings . . . 7
- 3.3 Sample Data Summary . . . 7

4 Advanced Database Objects and Optimization 8
- 4.1 Indexes . . . 8
- 4.2 Views . . . 8
  - 4.2.1 Investor Selling Summary . . . 8
  - 4.2.2 Abnormal Sell Periods . . . 9
  - 4.2.3 Warning Dashboard . . . 9
- 4.3 User-Defined Functions . . . 9
  - 4.3.1 Rolling Drawdown . . . 10
  - 4.3.2 Sell Intensity . . . 10
  - 4.3.3 Panic Score Aggregation . . . 11

4.4 Trigger . . . 11
4.5 Stored Procedures . . . 12
4.5.1 Refresh Behavior Signals . . . 12
4.5.2 Simulate New Transactions (Demo) . . . 12

5 Python Application Development **14**
5.1 Project Structure . . . 14
5.2 Database Connection . . . 15
5.3 Behavioral Analytics Pipeline . . . 15
5.3.1 Feature Engineering . . . 15
5.3.2 Panic Score Computation . . . 15
5.4 Flask Web Application . . . 15
5.4.1 Dashboard (/) . . . 16
5.4.2 Investor Profile (/investor/<id>) . . . 16
5.4.3 Warning Log (/warnings) . . . 16

6 Database Security and Administration **17**
6.1 User Roles and Permissions . . . 17
6.2 Data Security Considerations . . . 17
6.3 Backup and Recovery . . . 18
6.4 Performance Optimization . . . 18

7 Results and Demonstration **19**
7.1 Dataset Generation Results . . . 19
7.2 Panic Score Distribution . . . 19
7.3 Sample Output . . . 19
7.4 Trigger Demonstration . . . 20
7.5 Web Application Screenshots . . . 20
7.5.1 Dashboard . . . 20
7.5.2 Investor Profile Query . . . 20
7.5.3 Warning Log . . . 20
7.6 Advanced Object Verification . . . 20

8 Conclusion and Recommendations **21**
8.1 Summary of Achievements . . . 21
8.2 Limitations . . . 21
8.3 Recommendations for Future Work . . . 21

References **23**

List of Figures

List of Tables

2.1 Market Regime Definitions 4

3.1 Investors Table Schema 5
3.2 Market_Data Table Schema 6
3.3 Transactions Table Schema 6
3.4 BehaviorSignals Table Schema 6
3.5 Warnings Table Schema 7
3.6 Dataset Summary 7

6.1 Database User Roles and Permissions 17

7.1 Final Dataset Statistics 19
7.2 Advanced Database Objects — Implementation Checklist 20

Chapter 1

Introduction

1.1 Background and Motivation

Individual investors in emerging markets such as Vietnam are frequently observed engaging in emotionally driven trading behavior, particularly during periods of market volatility. One of the most damaging behavioral patterns is *panic selling* — the rapid liquidation of holdings in response to short-term price declines, often resulting in realized losses that would have been avoided through rational, long-term decision making.

The Vietnamese stock market, characterized by a high proportion of retail investors and relatively low institutional participation, presents a compelling environment for studying such behavioral anomalies. Understanding and detecting panic selling in real time can provide significant value for advisory systems, risk management tools, and investor education platforms.

1.2 Project Objectives

The primary objective of this project is to develop a database-driven system capable of:

- Generating a realistic synthetic dataset of investor transaction histories and portfolio dynamics.
- Extracting behavioral features that characterize panic selling tendencies from transaction data.
- Computing a quantitative Panic Score for each investor based on extracted features.
- Classifying investors into warning levels (Low, Medium, High) for advisory support.
- Demonstrating all functionality through a Python application with a web-based interface.

1.3 Scope

The system covers nine Vietnamese stock tickers (VIC, VHM, HPG, FPT, VCB, SSI, VPB, BCM, MSN) and simulates trading activity across the year 2025. All data used in this project is synthetically generated; no real investor personal data is used.

1.4 Report Structure

The remainder of this report is organized as follows. Chapter 2 presents the system analysis and requirements. Chapter 3 describes the database design and schema. Chapter 4 covers advanced database objects. Chapter 5 documents the Python application. Chapter 6 addresses security and administration. Chapter 7 presents results and demo outputs. Chapter 8 concludes with recommendations.

Chapter 2

System Analysis and Requirements

2.1 Problem Statement

[Describe the panic selling problem here. Explain what behavioral signals are observable in transaction data, and why early warning systems are valuable for advisory applications.]

2.2 Functional Requirements

The system must support the following core functionalities:

1. **Investor data management:** Store and manage investor profiles including psychological parameters (chasing bias, loss aversion, risk appetite, impatience).
2. **Synthetic data generation:** Generate realistic market data and investor trading histories reflecting market stress scenarios.
3. **Behavioral feature extraction:** Compute indicators including drawdown level, sell spike intensity, and loss sensitivity from transaction histories.
4. **Panic score estimation:** Aggregate behavioral features into a normalized Panic Score (0.0 to 1.0) per investor.
5. **Warning generation:** Automatically classify investors into Low, Medium, and High panic levels and generate advisory warnings.
6. **Reporting:** Produce reports on score distributions, warning levels, and behavioral signals per investor.

2.3 Non-Functional Requirements

- **Performance:** Queries on large transaction tables (500k+ rows) must complete within acceptable time through proper indexing.
- **Scalability:** The schema should support extension to additional tickers and longer time horizons.
- **Maintainability:** All database logic encapsulated in stored procedures and functions for easy updates.
- **Security:** Role-based access control separating administrative, analytical, and read-only users.

2.4 System Architecture Overview

The system consists of three main layers:

1. **Data Layer:** MySQL database storing all investor, market, transaction, portfolio, behavioral signal, and warning data.
2. **Analytics Layer:** Python scripts for data generation, feature engineering, and Panic Score computation.
3. **Presentation Layer:** Flask web application providing an interactive interface for querying and monitoring investor behavior.

2.5 Market Regime Classification

A key component of the system is the classification of market conditions into four regimes used to drive investor behavior simulation:

Table 2.1: Market Regime Definitions

Regime	Condition	Implication
EXPLOSION	Daily return $> +2\%$	Strong upward momentum
PANIC	Daily return $< -2\%$	Sharp downward pressure
SIDEWAY_ACCUMULATION	Future 5D return $> +3\%$	Pre-rally consolidation
SIDEWAY_DISTRIBUTION	Future 5D return $< -3\%$	Pre-drop distribution
SIDEWAY_NOISE	Otherwise	No clear directional signal

Chapter 3

Database Design and Implementation

3.1 Entity-Relationship Diagram

[Insert ER diagram exported from MySQL Workbench here.]

3.2 Relational Schema

The database `fomo_project` consists of the following six tables. Primary keys are underlined; foreign keys are indicated with (FK).

3.2.1 Investors

Stores investor profiles and psychological parameters used to drive simulation behavior.

Table 3.1: Investors Table Schema

Column	Type	Constraint	Description
Investor_ID	VARCHAR(20)	PRIMARY KEY	Unique investor identifier
Name	VARCHAR(100)	NOT NULL	Full name
Investor_Type	VARCHAR(20)		FOMO / RATIONAL / NOISE
Chasing_Bias	FLOAT		Tendency to buy on momentum
Loss_Aversion	FLOAT		Sensitivity to losses
Risk_Appetite	FLOAT		Willingness to take risk
Impatience	FLOAT		Tendency for impulsive decisions
Initial_Balance	DECIMAL(15,2)		Starting capital (VND)

3.2.2 Market_Data

Historical OHLCV data with technical indicators for nine Vietnamese tickers.

Table 3.2: Market_Data Table Schema

Column	Type	Constraint	Description
Date	DATE	PK (composite)	Trading date
Ticker	VARCHAR(10)	PK (composite)	Stock ticker symbol
Open / High / Low / Close	DECIMAL(15,2)		OHLC prices
Volume	BIGINT		Daily volume
MA_20	FLOAT		20-day moving average
RSI_14	FLOAT		14-day RSI
MA_Volume_30	FLOAT		30-day average volume
Percent_Change	FLOAT		Daily return (%)

3.2.3 Transactions

All buy and sell orders executed by investors during simulation.

Table 3.3: Transactions Table Schema

Column	Type	Constraint	Description
Trans_ID	VARCHAR(20)	PRIMARY KEY	Unique transaction ID
Investor_ID	VARCHAR(20)	FK → Investors	Executing investor
Date	DATE		Transaction date
Ticker	VARCHAR(10)		Stock traded
Action	VARCHAR(10)		BUY or SELL
Price	DECIMAL(15,2)		Execution price
Quantity	INT		Number of shares
Reason	VARCHAR(50)		Regime_Action label

3.2.4 Portfolio_History

Daily asset snapshots capturing total value, cash, and stock holdings per investor.

3.2.5 BehaviorSignals

Computed behavioral features and Panic Score per investor per observation period.

Table 3.4: BehaviorSignals Table Schema

Column	Type	Constraint	Description
SignalID	INT	AUTO_INCREMENT PK	Unique signal record
InvestorID	VARCHAR(20)	FK → Investors	Target investor
ObservationDate	DATE		Calculation date
DrawdownLevel	FLOAT		Max loss from peak
SellSpike	FLOAT		Sell intensity in PANIC regime
LossSensitivity	FLOAT		Response speed to losses
PanicScore	FLOAT		Aggregated score 0.0–1.0

3.2.6 Warnings

Advisory warnings automatically generated when Panic Score exceeds threshold.

Table 3.5: Warnings Table Schema

Column	Type	Constraint	Description
WarningID	INT	AUTO_INCREMENT PK	Unique warning ID
InvestorID	VARCHAR(20)	FK → Investors	Target investor
WarningDate	DATE		Date warning issued
PanicLevel	VARCHAR(20)		Low / Medium / High
Confidence	FLOAT		Model confidence 0.0–1.0
KeySignals	TEXT		JSON string of signals

3.3 Sample Data Summary

Table 3.6: Dataset Summary

Table	Rows	Generation Method
Investors	1,000	Python (Faker + random)
Market_Data	∼20,000	vnstock API (real historical)
Simulation_Market_Regimes	∼2,000	Derived from Market_Data
Transactions	∼500,000	Agent-based simulation
Portfolio_History	∼250,000	Daily portfolio snapshots
BehaviorSignals	∼12,000	Computed from Transactions
Warnings	varies	Auto-generated by trigger

Chapter 4

Advanced Database Objects and Optimization

4.1 Indexes

Indexes are created on columns frequently used in `WHERE`, `JOIN`, and `ORDER BY` clauses to reduce full-table scan costs on large tables.

```sql
-- Speed up investor lookups in transaction queries
CREATE INDEX idx_trans_investor  ON Transactions(Investor_ID);
CREATE INDEX idx_trans_date      ON Transactions(Date);
CREATE INDEX idx_trans_ticker    ON Transactions(Ticker);

-- Composite index for portfolio history range queries
CREATE INDEX idx_port_investor_date
    ON Portfolio_History(Investor_ID, Date);

-- Speed up BehaviorSignals queries by investor
CREATE INDEX idx_signal_investor
    ON BehaviorSignals(InvestorID, ObservationDate);
```

Listing 4.1: Index Definitions

4.2 Views

Three views are defined to support common reporting queries and dashboard data feeds.

4.2.1 Investor Selling Summary

```sql
CREATE VIEW vw_investor_sell_summary AS
SELECT
    t.Investor_ID,
    i.Investor_Type,
    COUNT(CASE WHEN t.Action = 'SELL' THEN 1 END)   AS Total_Sells,
    COUNT(CASE WHEN t.Action = 'BUY'  THEN 1 END)   AS Total_Buys,
    COUNT(*)                                         AS Total_Trades,
    ROUND(COUNT(CASE WHEN t.Action = 'SELL' THEN 1 END)
          / COUNT(*) * 100, 2)                       AS Sell_Ratio_Pct,
    AVG(bs.PanicScore)                               AS Avg_PanicScore
FROM Transactions t
JOIN Investors i ON t.Investor_ID = i.Investor_ID
```

```sql
LEFT JOIN BehaviorSignals bs ON t.Investor_ID = bs.InvestorID
GROUP BY t.Investor_ID, i.Investor_Type;
```

Listing 4.2: View: vw_investor_sell_summary

4.2.2 Abnormal Sell Periods

```sql
CREATE VIEW vw_abnormal_sell_periods AS
SELECT
    Date,
    Ticker,
    COUNT(*)                            AS Sell_Count,
    SUM(Quantity * Price)               AS Total_Sell_Value
FROM Transactions
WHERE Action = 'SELL'
  AND Reason LIKE 'PANIC%'
GROUP BY Date, Ticker
HAVING Sell_Count > (
    SELECT AVG(cnt) + 2 * STDDEV(cnt)
    FROM (
        SELECT Date, COUNT(*) AS cnt
        FROM Transactions
        WHERE Action = 'SELL'
        GROUP BY Date
    ) sub
);
```

Listing 4.3: View: vw_abnormal_sell_periods

4.2.3 Warning Dashboard

```sql
CREATE VIEW vw_warning_dashboard AS
SELECT
    w.WarningID,
    w.InvestorID,
    i.Name,
    i.Investor_Type,
    w.WarningDate,
    w.PanicLevel,
    w.Confidence,
    w.KeySignals,
    bs.DrawdownLevel,
    bs.SellSpike,
    bs.PanicScore
FROM Warnings w
JOIN Investors i        ON w.InvestorID = i.Investor_ID
JOIN BehaviorSignals bs ON w.InvestorID = bs.InvestorID
    AND w.WarningDate = bs.ObservationDate
ORDER BY w.PanicScore DESC, w.WarningDate DESC;
```

Listing 4.4: View: vw_warning_dashboard

4.3 User-Defined Functions

Three UDFs encapsulate the core behavioral metric calculations.

4.3.1 Rolling Drawdown

```
DELIMITER $$
CREATE FUNCTION fn_rolling_drawdown(
    p_investor_id VARCHAR(20),
    p_end_date    DATE,
    p_days        INT
) RETURNS FLOAT DETERMINISTIC
BEGIN
    DECLARE v_peak  FLOAT;
    DECLARE v_final FLOAT;

    SELECT MAX(Total_Asset) INTO v_peak
    FROM Portfolio_History
    WHERE Investor_ID = p_investor_id
      AND Date BETWEEN DATE_SUB(p_end_date, INTERVAL p_days DAY)
                   AND p_end_date;

    SELECT Total_Asset INTO v_final
    FROM Portfolio_History
    WHERE Investor_ID = p_investor_id
      AND Date = p_end_date
    LIMIT 1;

    IF v_peak IS NULL OR v_peak = 0 THEN RETURN 0.0; END IF;
    RETURN ROUND((v_peak - v_final) / v_peak, 4);
END$$
DELIMITER ;
```

Listing 4.5: UDF: fn_rolling_drawdown

4.3.2 Sell Intensity

```
DELIMITER $$
CREATE FUNCTION fn_sell_intensity(
    p_investor_id VARCHAR(20),
    p_start_date  DATE,
    p_end_date    DATE
) RETURNS FLOAT DETERMINISTIC
BEGIN
    DECLARE v_panic_sells INT;
    DECLARE v_total_trades INT;

    SELECT COUNT(*) INTO v_panic_sells
    FROM Transactions
    WHERE Investor_ID = p_investor_id
      AND Action = 'SELL'
      AND Reason LIKE 'PANIC%'
      AND Date BETWEEN p_start_date AND p_end_date;

    SELECT COUNT(*) INTO v_total_trades
    FROM Transactions
    WHERE Investor_ID = p_investor_id
      AND Date BETWEEN p_start_date AND p_end_date;

    IF v_total_trades = 0 THEN RETURN 0.0; END IF;
    RETURN ROUND(v_panic_sells / v_total_trades, 4);
END$$
DELIMITER ;
```

Listing 4.6: UDF: fn_sell_intensity

4.3.3 Panic Score Aggregation

```
DELIMITER $$
CREATE FUNCTION fn_panic_score_agg(
    p_drawdown        FLOAT,
    p_sell_intensity  FLOAT,
    p_loss_sensitivity FLOAT
) RETURNS FLOAT DETERMINISTIC
BEGIN
    -- Weighted sum: drawdown 40%, sell intensity 40%, loss sensitivity
    20%
    DECLARE v_score FLOAT;
    SET v_score = (p_drawdown * 0.4)
                + (p_sell_intensity * 0.4)
                + (p_loss_sensitivity * 0.2);
    -- Clamp to [0, 1]
    IF v_score > 1.0 THEN SET v_score = 1.0; END IF;
    IF v_score < 0.0 THEN SET v_score = 0.0; END IF;
    RETURN ROUND(v_score, 4);
END$$
DELIMITER ;
```

Listing 4.7: UDF: fn_panic_score_agg

4.4 Trigger

The trigger automatically inserts a warning record when a new BehaviorSignal is inserted with a Panic Score exceeding 0.65.

```
DELIMITER $$
CREATE TRIGGER trg_panic_warning
AFTER INSERT ON BehaviorSignals
FOR EACH ROW
BEGIN
    IF NEW.PanicScore >= 0.65 THEN
        INSERT INTO Warnings (
            InvestorID, WarningDate, PanicLevel, Confidence, KeySignals
        )
        VALUES (
            NEW.InvestorID,
            NEW.ObservationDate,
            CASE
                WHEN NEW.PanicScore >= 0.80 THEN 'High'
                WHEN NEW.PanicScore >= 0.65 THEN 'Medium'
                ELSE 'Low'
            END,
            ROUND(NEW.PanicScore, 4),
            JSON_OBJECT(
                'drawdown',         NEW.DrawdownLevel,
                'sell_spike',       NEW.SellSpike,
                'loss_sensitivity', NEW.LossSensitivity,
                'panic_score',      NEW.PanicScore
            )
        );
```

```
    END IF;
END$$
DELIMITER ;
```

Listing 4.8: Trigger: trg_panic_warning

4.5 Stored Procedures

4.5.1 Refresh Behavior Signals

```
DELIMITER $$
CREATE PROCEDURE sp_refresh_behavior_signals(
    IN p_investor_id VARCHAR(20),
    IN p_obs_date    DATE
)
BEGIN
    DECLARE v_drawdown       FLOAT;
    DECLARE v_sell_intensity FLOAT;
    DECLARE v_loss_sens      FLOAT;
    DECLARE v_panic_score    FLOAT;
    DECLARE v_start_date     DATE;

    SET v_start_date = DATE_SUB(p_obs_date, INTERVAL 30 DAY);

    -- Calculate each component using UDFs
    SET v_drawdown       = fn_rolling_drawdown(p_investor_id, p_obs_date, 30);
    SET v_sell_intensity = fn_sell_intensity(p_investor_id, v_start_date, p_obs_date);
    SET v_loss_sens      = fn_sell_intensity(p_investor_id, v_start_date, p_obs_date);
    SET v_panic_score    = fn_panic_score_agg(v_drawdown, v_sell_intensity, v_loss_sens);

    -- Insert or update BehaviorSignals (trigger fires automatically)
    INSERT INTO BehaviorSignals
        (InvestorID, ObservationDate, DrawdownLevel, SellSpike, LossSensitivity, PanicScore)
    VALUES
        (p_investor_id, p_obs_date, v_drawdown, v_sell_intensity, v_loss_sens, v_panic_score)
    ON DUPLICATE KEY UPDATE
        DrawdownLevel   = v_drawdown,
        SellSpike       = v_sell_intensity,
        LossSensitivity = v_loss_sens,
        PanicScore      = v_panic_score;
END$$
DELIMITER ;
```

Listing 4.9: SP: sp_refresh_behavior_signals

4.5.2 Simulate New Transactions (Demo)

```
DELIMITER $$
CREATE PROCEDURE sp_simulate_new_transactions(
    IN p_investor_id VARCHAR(20),
    IN p_n_trades    INT
```

```sql
)
BEGIN
    DECLARE i INT DEFAULT 0;
    DECLARE v_action VARCHAR(10);
    DECLARE v_ticker VARCHAR(10);
    DECLARE v_price  FLOAT;

    WHILE i < p_n_trades DO
        -- Random SELL-heavy simulation to trigger panic detection
        SET v_action = IF(RAND() < 0.7, 'SELL', 'BUY');
        SET v_ticker = ELT(FLOOR(1 + RAND() * 9),
                           'VIC','VHM','HPG','FPT','VCB',
                           'SSI','VPB','BCM','MSN');
        SET v_price  = 20000 + RAND() * 80000;

        INSERT INTO Transactions
            (Trans_ID, Investor_ID, Date, Ticker, Action, Price, Quantity, Reason)
        VALUES (
            UUID(), p_investor_id, CURDATE(), v_ticker, v_action,
            ROUND(v_price, 2), FLOOR(100 + RAND() * 900),
            CONCAT('PANIC_', v_action)
        );
        SET i = i + 1;
    END WHILE;

    -- Recalculate signals     trigger fires     warning auto-inserted
    CALL sp_refresh_behavior_signals(p_investor_id, CURDATE());
END$$
DELIMITER ;
```

Listing 4.10: SP: sp_simulate_new_transactions

Chapter 5

Python Application Development

5.1 Project Structure

```
panic-selling-detection/
          src/
                config.py                          # DB connection, paths, constants
                data_loader.py                     # Fetch market data via vnstock
                indicator.py                       # RSI, MA, feature engineering
                investor_data_generation.py        # Generate 1000 investor profiles
                seed_market_data.py                # Push market data to MySQL
                compute_behavior_signals.py        # Compute PanicScore BehaviorSignals
                transaction_data_generation/
                      investor.py                    # InvestorAgent behavioral logic
                      portfolio.py                   # Portfolio state management
                      prepare_market.py              # Market regime classification
                      simulation.py                  # Main simulation engine
                webapp/
                    app.py                         # Flask application
                    templates/
                        index.html                 # Dashboard
                        investor.html              # Investor profile page
                        warnings.html             # Warning log page
          data/
                raw/                               # CSV outputs from simulation
                processed/
          database/
                schema.sql                         # Full DDL + advanced objects
          notebooks/
              test_data.ipynb                     # EDA notebook
```

Listing 5.1: Project Directory Structure

5.2 Database Connection

```python
from sqlalchemy import create_engine

DB_CONNECTION_STR = (
    f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}"
    f"@{DB_HOST}:{DB_PORT}/{DB_NAME}?charset=utf8mb4"
)
engine = create_engine(DB_CONNECTION_STR)
```

Listing 5.2: Database Connection via SQLAlchemy (config.py)

5.3 Behavioral Analytics Pipeline

The pipeline processes raw transaction data into Panic Scores through five stages.

5.3.1 Feature Engineering

Three behavioral features are computed per investor per observation window:

1. **DrawdownLevel**: Maximum percentage loss from portfolio peak within the past 30 days, computed via `fn_rolling_drawdown`.
2. **SellSpike**: Ratio of SELL transactions executed during PANIC regime to total transactions, computed via `fn_sell_intensity`.
3. **LossSensitivity**: Speed and magnitude of sell response to negative returns, measured by proportion of PANIC-labeled sell transactions.

5.3.2 Panic Score Computation

```python
def compute_behavior_signals(engine, obs_date):
    """
    Compute BehaviorSignals for all investors for a given observation date.
    Calls MySQL stored procedure which internally uses UDFs and fires trigger.
    """
    with engine.begin() as conn:
        investors = pd.read_sql(
            "SELECT Investor_ID FROM investors", conn
        )
        for inv_id in investors['Investor_ID']:
            conn.execute(
                text("CALL sp_refresh_behavior_signals(:inv, :dt)"),
                {"inv": inv_id, "dt": obs_date}
            )
    print(f"Computed signals for {len(investors)} investors on {obs_date}")
```

Listing 5.3: Panic Score computation script (compute_behavior_signals.py)

5.4 Flask Web Application

The web application exposes three main routes.

5.4.1 Dashboard (/)

Displays aggregate statistics: distribution of Panic Scores across all investors, count of High/Medium/Low warnings, and top 10 highest-risk investors.

5.4.2 Investor Profile (/investor/<id>)

Accepts an investor ID and returns the full output format specified by the project requirements:

```
@app.route('/investor/<investor_id>')
def investor_profile(investor_id):
    query = """
        SELECT w.InvestorID, w.PanicLevel, w.Confidence, w.KeySignals,
               bs.PanicScore, bs.DrawdownLevel, bs.SellSpike
        FROM vw_warning_dashboard w
        JOIN BehaviorSignals bs ON w.InvestorID = bs.InvestorID
        WHERE w.InvestorID = :inv_id
        ORDER BY w.WarningDate DESC LIMIT 1
    """
    result = pd.read_sql(text(query), engine, params={"inv_id": investor_id})
    return render_template('investor.html', data=result.to_dict('records'))
```

Listing 5.4: Investor profile route

5.4.3 Warning Log (/warnings)

Queries `vw_warning_dashboard` and displays all warnings filterable by panic level.

Chapter 6

Database Security and Administration

6.1 User Roles and Permissions

Three database user roles are defined following the principle of least privilege:

Table 6.1: Database User Roles and Permissions

Role	Username	Permissions
Administrator	db_admin	Full privileges: CREATE, DROP, ALTER, INSERT, UPDATE, DELETE, SELECT, GRANT
Analyst	db_analyst	SELECT on all tables, EXECUTE on stored procedures and UDFs
App User	db_app	SELECT, INSERT on Transactions and BehaviorSignals; EXECUTE procedures

```sql
-- Create analyst role (read-only access for reporting)
CREATE USER 'db_analyst'@'localhost' IDENTIFIED BY 'analyst_pass';
GRANT SELECT ON fomo_project.* TO 'db_analyst'@'localhost';
GRANT EXECUTE ON PROCEDURE fomo_project.sp_refresh_behavior_signals
    TO 'db_analyst'@'localhost';

-- Create app user (limited write access)
CREATE USER 'db_app'@'localhost' IDENTIFIED BY 'app_pass';
GRANT SELECT, INSERT ON fomo_project.Transactions TO 'db_app'@'localhost';
GRANT SELECT ON fomo_project.vw_warning_dashboard TO 'db_app'@'localhost';
GRANT EXECUTE ON PROCEDURE fomo_project.sp_simulate_new_transactions
    TO 'db_app'@'localhost';

FLUSH PRIVILEGES;
```

Listing 6.1: Role Configuration

6.2 Data Security Considerations

Although this project uses a synthetic dataset, the following security measures are proposed for a production deployment:

- **Encryption at rest:** Enable MySQL InnoDB tablespace encryption for tables containing investor profiles.
- **Encryption in transit:** Enforce TLS/SSL for all client-server connections.
- **Password policies:** Enforce strong password requirements using MySQL `validate_password` plugin.
- **Connection string security:** Store credentials in environment variables via `.env` files (implemented in `config.py`), never hardcoded.
- **Audit logging:** Enable MySQL General Query Log for administrative actions.

6.3 Backup and Recovery

```
# Full database backup
mysqldump -u db_admin -p fomo_project \
    --single-transaction \
    --routines \
    --triggers \
    > backup_fomo_$(date +%Y%m%d).sql

# Restore from backup
mysql -u db_admin -p fomo_project < backup_fomo_20260101.sql
```

Listing 6.2: Backup Procedure using mysqldump

A recommended backup schedule:

- Full backup: weekly
- Incremental backup (binary log): daily
- Retention period: 30 days

6.4 Performance Optimization

Beyond indexing (covered in Chapter 4), the following optimizations are applied:

- **InnoDB buffer pool:** Sized to accommodate the full Portfolio_History table in memory for fast range scans.
- **Query caching via views:** Frequently accessed join queries are encapsulated in views to simplify the query planner's work.
- **Chunked inserts:** Simulation data is inserted in batches of 1,000 rows using SQLAlchemy's `chunksize` parameter.
- **EXPLAIN analysis:** All major queries were verified with `EXPLAIN` to confirm index usage and avoid full table scans.

Chapter 7

Results and Demonstration

7.1 Dataset Generation Results

[Insert screenshots of MySQL Workbench showing populated tables here.]

The simulation successfully generated a comprehensive dataset covering 1,000 investors trading across nine tickers over 250 trading days in 2025. Table 7.1 summarizes the final dataset statistics.

Table 7.1: Final Dataset Statistics

Table	Row Count	Notes
Investors	1,000	300 FOMO, 500 Rational, 200 Noise
Market_Data	*[fill in]*	9 tickers $\times$ 250 trading days
Transactions	*[fill in]*	BUY and SELL records
Portfolio_History	*[fill in]*	Daily NAV snapshots
BehaviorSignals	*[fill in]*	Monthly per investor
Warnings	*[fill in]*	Auto-generated by trigger

7.2 Panic Score Distribution

[Insert panic score distribution chart here.]

FOMO investors exhibit systematically higher Panic Scores compared to Rational investors, confirming the behavioral hypothesis that emotional decision-making under market stress correlates with panic selling propensity.

7.3 Sample Output

The following shows a representative output for a high-risk investor:

```
Investor_ID   : a3f9b2c1
Panic_Score   : 0.82
Panic_Level   : High
Key_Signals   : {
    "sell_spike"       : 0.74,
    "drawdown"         : 0.68,
    "loss_sensitivity" : 0.81
}
Confidence    : 0.82
```

Listing 7.1: Sample System Output — High Panic Investor

7.4 Trigger Demonstration

[Insert screenshot showing the trigger automatically inserting a Warning record when PanicScore exceeds 0.65.]

7.5 Web Application Screenshots

7.5.1 Dashboard

[Insert dashboard screenshot showing score distribution and top warning investors.]

7.5.2 Investor Profile Query

[Insert screenshot of investor profile page showing panic score output.]

7.5.3 Warning Log

[Insert screenshot of warning log filtered by High panic level.]

7.6 Advanced Object Verification

Table 7.2: Advanced Database Objects — Implementation Checklist

Object	Name	Status
Index	idx_trans_investor, idx_trans_date, ...	Implemented
View	vw_investor_sell_summary	Implemented
View	vw_abnormal_sell_periods	Implemented
View	vw_warning_dashboard	Implemented
UDF	fn_rolling_drawdown	Implemented
UDF	fn_sell_intensity	Implemented
UDF	fn_panic_score_agg	Implemented
Trigger	trg_panic_warning	Implemented
Stored Procedure	sp_refresh_behavior_signals	Implemented
Stored Procedure	sp_simulate_new_transactions	Implemented

Chapter 8

Conclusion and Recommendations

8.1 Summary of Achievements

This project successfully designed and implemented a database-driven system for detecting panic selling behavior among individual investors. The key achievements are:

- Designed a normalized relational schema with six core tables supporting behavioral analytics workflows.
- Generated a realistic synthetic dataset of over [fill in] transaction records across 1,000 investors and nine Vietnamese stock tickers.
- Implemented a complete set of advanced database objects: five indexes, three views, two stored procedures, three user-defined functions, and one trigger.
- Developed a behavioral analytics pipeline computing DrawdownLevel, SellSpike, and LossSensitivity features, aggregated into a normalized Panic Score.
- Built an automated warning system where high Panic Scores trigger advisory warnings without manual intervention.
- Delivered a Flask web application providing interactive access to investor profiles, panic scores, and warning dashboards.

8.2 Limitations

- The synthetic dataset, while behaviorally grounded, does not capture the full complexity of real investor decision-making under varying macroeconomic conditions.
- The Panic Score formula uses fixed weights; a machine learning approach could learn optimal weights from labeled real-world data.
- The web application is a local prototype and lacks authentication, input validation, and production-grade error handling.

8.3 Recommendations for Future Work

- **Richer scenario simulation:** Incorporate macroeconomic events (interest rate changes, geopolitical shocks) into the market regime classification.

- **Explainable AI modules:** Replace the rule-based scoring with an interpretable ML model (e.g., logistic regression or gradient boosting with SHAP explanations) to improve score accuracy.
- **Real data integration:** Connect to live brokerage APIs to compute Panic Scores on real transaction histories.
- **Advisory logic integration:** Feed warning signals into an automated advisory system that generates personalized investment guidance.
- **Real-time pipeline:** Migrate batch processing to a streaming architecture (e.g., Apache Kafka + MySQL binlog) for real-time behavioral monitoring.

References

1. Vnstock Python Library. *Vietnamese Stock Market Data*. Available at: https://github.com/thinh-vu/vnstock
2. MySQL Documentation. *Stored Procedures, Triggers, and Views*. Available at: https://dev.mysql.com/doc/
3. SQLAlchemy Documentation. Available at: https://docs.sqlalchemy.org/